6주차 : 딥러닝 파이토치 교과서 6장-Part2(6.1.5~)

6.1.5 : ResNet

- ResNet : VGG19 구조를 뼈대로 하며, 합성곱층들을 추가해서 깊게 만든 후 숏컷들을 추가한 것
  - 네트워크 깊이가 깊다고 해서 무조건 성능이 좋아지는 것이 아님
  - 깊어진 신경망을 효과적으로 학습하기 위해 레지듀얼 블록(residual block)을 도입함
  - 레지듀얼 블록 : 기울기가 잘 전파되도록 일종의 숏컷(short cut, skip connection)을 만들어 줌

    → 출력값을 F(x) + x 꼴로 바꿈 : 모델이 학습해야 하는 잔차는 H(x) - x (=정답과 입력의 차이, residual)에 불과함 ⇒ 이미 알고 있는 정보 x는 숏컷으로 넘겨받고, 메인 경로는 약간의 수정사항(residual)만 학습하면 되므로 훨씬 난이도가 낮아짐.
  - 블록 : 계층의 묶음 → 합성곱층을 하나의 블록으로 묶은 것

    사진과 같이 색색의 레지듀얼 블록을 여러 개 쌓은 것 → ResNet
  - 병목 블록 : 파라미터 수가 감소하는 효과를 줄 수 있음
    - 계층이 깊어질수록 파라미터 증가
      → overfitting 발생 가능성 증가, 연산량 및 메모리 사용량 증가, 학습 난이도 증가
      ⇒ 이를 해결하기 위해 병목 블록을 둠
    - 병목(Bottleneck) 구조가 도움이 되는 이유

    > *"층이 늘어나 파라미터가 늘어나면 다 제 역할이 있는 것 아닌가?"*

    맞습니다! **층(Depth)을 깊게 쌓아서 다양한 특징을 배우는 역할**은 꼭 필요합니다.

    문제는 연산량과 파라미터가 폭발하는 주범이 '층의 깊이'가 아니라 '채널(Channel)의 개수'라는 점입니다.

    ⇒ 1×1 합성곱(Convolution)을 이용한 병목의 원리

ResNet의 병목 블록(Bottleneck Block)은 **3개의 층**으로 구성됩니다.

```
[입력: 256 채널]
       ↓
 (1) 1x1 Conv  ⟶  채널 압축 (256 → 64)   [압축!]
       ↓
 (2) 3x3 Conv  ⟶  특징 추출 (64 → 64)    [중요 연산]
       ↓
 (3) 1x1 Conv  ⟶  채널 복원 (64 → 256)   [복원!]
       ↓
[출력: 256 채널]
```

1. **1×1 Conv (압축):** 256개나 되는 채널 중 중복되거나 덜 중요한 정보를 줄여 핵심만 **64개로 차원을 축소**합니다.
2. **3×3 Conv (연산):** 연산 비용이 많이 드는 3x3 공간 연산을 **가벼워진 64개 채널에 대해서만 수행**합니다.
3. **1×1 Conv (복원):** 연산이 끝난 후 다시 원래의 **256개 채널 형태로 늘려줍니다.**

- 정보가 손실되지 않나요?

  ⇒ 채널을 줄일 때 핵심 정보만 남기고 불필요한 정보(Noise/Redundancy)를 제거하는 효과가 있습니다. **"무작정 많이 계산하는 것"보다 "핵심 요약본으로 효율적으로 계산하는 것"이 성능과 속도 모두에서 유리**하기 때문에 병목 구조를 사용합니다.

- 아이덴티티 매핑 (identitiy mapping) (혹은 숏컷, 스킵 연결) → 사진 속 + 동그라미
  : 입력 x가 어떤 함수를 통과하더라도 다시 x라는 형태로 출력되도록 함
  - 아이덴티티 매핑이 필요한 이유?
    : 깊은 모델이 적어도 얕은 모델보다 못해지지 않도록 안전성을 확보하고, 학습 시 기울기를 최전방까지 전달하는 깨끗한 통로를 확보하기 위함
- 다운샘플 (downsample) : 특성맵의 크기를 줄이기 위한 것 → 풀링과 같은 역할
  - 네트워크가 깊어지며 특성 맵 크기는 줄이고 채널을 늘리는 과정에서, 메인 경로 연산 결과와 원본 입력 x의 행렬 차원(모양)이 달라짐
    ⇒ 차원 불일치를 해결하기 위해 숏컷 경로에 지나가는 x의 모양을 변형해주는 작업이 필요
- 합성곱 블록 : 입출력 차원이 다를 때, 다운샘플을 적용하여 차원을 맞춰준 뒤 더하는 블록

        - 차원 변화 : 입출력의 크기나 채널 수가 다름
        - 숏컷 연산 : 차원을 맞추기 위한 1 * 1 Conv(다운샘플) 적용
        - 사용 위치 : 네트워크에서 특성 맵 크기를 줄이고 채널을 늘리는 전환 지점
    - 아이덴티티 블록 : 아무런 가공 없이 x를 그냥 전달하여 덧셈을 수행하는 블록
      → 숏컷 경로에 아이덴티티 매핑 적용
        - 차원 변화 : 입출력의 크기와 채널 수가 같음
        - 숏컷 연산 : 아무 연산 없이 그대로 전달(x)
        - 사용 위치 : 특성 맵 크기가 유지되는 구간 내부

ResNet 예제

```
# 코드 6-73 : 기본 블록 정의

class BasicBlock(nn.Module):
    expansion = 1

    def __init__(self, in_channels, out_channels, stride = 1,
downsample = False):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, ker
nel_size = 3,
                               stride = stride, padding = 1,
bias = False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, ke
rnel_size = 3,
                               stride = 1, padding = 1, bias
= False)
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace = True)

        if downsample: # 다운샘플 적용
            conv = nn.Conv2d(in_channels, out_channels, kerne
```

```
l_size = 1,
                                stride = stride, bias = False)
            bn = nn.BatchNorm2d(out_channels)
            downsample = nn.Sequential(conv, bn)
        else:
            downsample = None
        self.downsample = downsample

    def forward(self, x):
        i = x
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.conv2(x)
        x = self.bn2(x)

        if self.downsample is not None:
            i = self.downsample(i)

        x += i # 아이덴티티 매핑 적용
        x = self.relu(x)

        return x
```

```
# 코드 6-74 : 병목 블록 정의

class Bottleneck(nn.Module):
    expansion = 4 # ResNet에서 병목 블록을 정의하기 위한 하이퍼파라
미터

    def __init__(self, in_channels, out_channels, stride = 1,
downsample = False):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, ker
nel_size = 1, stride = 1, bias = False) # 1*1 합성곱층
```

```
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, ke
rnel_size = 3, stride = stride, padding = 1, bias = False) #
3*3 합성곱층
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.conv3 = nn.Conv2d(out_channels, self.expansion *
out_channels, kernel_size = 1,
                               stride = 1, bias = False)
        self.bn3 = nn.BatchNorm2d(self.expansion * out_channe
ls)
        self.relu = nn.ReLU(inplace = True)

        if downsample:
            conv = nn.Conv2d(in_channels, self.expansion * ou
t_channels, kernel_size = 1,
                             stride = stride, bias = False) #
1*1 합성곱층
            bn = nn.BatchNorm2d(self.expansion * out_channel
s) # 다음 계층의 입력 채널 수와 일치하도록
            downsample = nn.Sequential(conv, bn)
        else:
            downsample = None
        self.downsample = downsample

    def forward(self, x):
        i = x
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.conv2(x)
        x = self.bn2(x)
        x = self.relu(x)
        x = self.conv3(x)
        x = self.bn3(x)

        if self.downsample is not None:
```

```
            i = self.downsample(i)

        x += i
        x = self.relu(x)

        return x
```

- 기본 블록과 병목 블록의 차이
  - 기본 블록 : 3*3 합성곱층 두 개를 가짐
  - 병목블록 : 1*1, 3*3, 1*1 합성곱층의 구조를 가짐
  - 기본 블록을 병목 블록으로 변경하는 이유
    - 계층을 더 깊게 쌓으면서 계산에 대한 비용을 줄일 수 있기 때문
    - 계층이 많아짐 → 활성화 함수가 기존보다 더 많이 포함됨 ⇒ 더 많은 비선형성을 처리할 수 있음 (다양한 입력 데이터에 대한 처리가 가능)

```
# 코드 6-75 : ResNet 모델 네트워크

class ResNet(nn.Module):
    def __init__(self, config, output_dim, zero_init_residual
=False):
        super().__init__()

        block, n_blocks, channels = config
        self.in_channels = channels[0]
        assert len(n_blocks) == len(channels) == 4 # 블록 크기
= 채널 크기 = 4

        self.conv1 = nn.Conv2d(3, self.in_channels, kernel_si
ze = 7, stride = 2, padding = 3, bias = False)
        self.bn1 = nn.BatchNorm2d(self.in_channels)
        self.relu = nn.ReLU(inplace = True)
        self.maxpool = nn.MaxPool2d(kernel_size = 3, stride =
2, padding = 1)
```

```
        self.layer1 = self.get_resnet_layer(block, n_blocks
[0], channels[0])
        self.layer2 = self.get_resnet_layer(block, n_blocks
[1], channels[1], stride = 2)
        self.layer3 = self.get_resnet_layer(block, n_blocks
[2], channels[2], stride = 2)
        self.layer4 = self.get_resnet_layer(block, n_blocks
[3], channels[3], stride = 2)

        self.avgpool = nn.AdaptiveAvgPool2d((1,1))
        self.fc = nn.Linear(self.in_channels, output_dim)

        if zero_init_residual: # 마지막 Batch Normalization을
0으로 초기화 -> 다음 레지듀얼 분기를 0에서 시작할 수 있도록 함
            for m in self.modules():
                if isinstance(m, Bottleneck):
                    nn.init.constant_(m.bn3.weight, 0)
                elif isinstance(m, BasicBlock):
                    nn.init.constant_(m.bn2.weight, 0)

    def get_resnet_layer(self, block, n_blocks, channels, str
ide = 1): # 블록을 추가하기 위한 함수
        layers = []
        if self.in_channels != block.expansion * channels: #
다르면
            downsample = True # 다운샘플 적용
        else:
            downsample = False

        layers.append(block(self.in_channels, channels, strid
e, downsample)) # 계층 추가시 다운샘플 적용 유무까지 함께 전달

        for i in range(1, n_blocks):
            layers.append(block(block.expansion * channels, c
hannels))
```

```
        self.in_channels = block.expansion * channels
        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.maxpool(x)
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.layer4(x)
        x = self.avgpool(x)
        h = x.view(x.shape[0], -1)
        x = self.fc(h)
        return x, h
```

```
# 코드 6-76 : ResNetConfig 정의

ResNetConfig = namedtuple('ResNetConfig', ['block', 'n_block
s', 'channels'])

# 코드 6-77 : 기본 블록을 사용하여 ResNetConfig 정의

resnet18_config = ResNetConfig(block = BasicBlock,
                               n_blocks = [2,2,2,2],
                               channels = [64, 128, 256, 51
2])

resnet34_config = ResNetConfig(block = BasicBlock,
                               n_blocks = [3,4,6,3],
                               channels = [64, 128, 256, 51
2])
```

```
# 코드 6-78 : 병목 블록을 사용하여 ResNetConfig 정의

resnet50_config = ResNetConfig(block = Bottleneck,
                               n_blocks = [3, 4, 6, 3],
                               channels = [64, 128, 256, 51
2])

resnet101_config = ResNetConfig(block = Bottleneck,
                                n_blocks = [3, 4, 23, 3],
                                channels = [64, 128, 256, 51
2])

resnet152_config = ResNetConfig(block = Bottleneck,
                                n_blocks = [3, 8, 36, 3],
                                channels = [64, 128, 256, 51
2])

# 코드  6-81 : ResNet50 Config를 사용한 ResNet 모델 사용

OUTPUT_DIM = 2
model = ResNet(resnet50_config, OUTPUT_DIM)
print(model)
```

- `ResNetConfig`는 "ResNet 모델을 어떻게 지을지 적어둔 청사진(설계도)"입니다. 설계도를 만들어두면 코드 한 줄로 **ResNet-18부터 ResNet-152까지 손쉽게 찍어낼 수 있고, 모델을 저장하거나 불러올 때 완벽하게 구조를 재현**할 수 있습니다.
- 직접 네트워크를 정의할 수도 있지만, 사전 정의된 네트워크를 불러와 사용할 수도 있다.

```
# 코드 6-82 : 옵티마이저와 손실 함수 정의

optimizer = optim.Adam(model.parameters(), lr=1e-7)
criterion = nn.CrossEntropyLoss()
```

```
model = model.to(device)
criterion = criterion.to(device)
```

```
# 코드 6-83 : 모델 학습 정확도 측정 함수 정의

def calculate_topk_accuracy(y_pred, y, k = 2):
    with torch.no_grad():
        batch_size = y.shape[0]
        _, top_pred = y_pred.topk(k, 1) # torch. argmax와 같은 효과 - 주어진 텐서에서 가장 큰 값의 index를 얻기 위해 사용
        top_pred = top_pred.t() # t의 의미 : 차원 0과 1을 tranpose 하겠다.
        correct = top_pred.eq(y.view(1, -1).expand_as(top_pred)) # 텐서를 비교하는 함수
        correct_1 = correct[:1].reshape(-1).float().sum(0, keepdim = True)
        correct_k = correct[:k].reshape(-1).float().sum(0, keepdim = True)
        acc_1 = correct_1 / batch_size
        acc_k = correct_k / batch_size
    return acc_1, acc_k
```

- 텐서를 비교하는 함수
  - 사용 예시 : torch.eq(비교 대상 텐서, 비교할 텐서)
  - 텐서가 서로 같은지 비교 : torch.eq
  - 텐서가 서로 다른지 비교 : torch.ne
  - 텐서가 크거나 같은지를 비교 : torch.ge

6.2 : 객체 인식을 위한 신경망

- 객체 인식 (object detection) : 이미지나 영상 내에 있는 객체를 식별하는 컴퓨터 비전 기술
  → 객체 인식 = 여러 가지 객체에 대한 분류 + 객체의 위치 정보를 파악하는 위치 검출
- 객체 인식 알고리즘 → 1단계 객체 인식과 2단계 객체 인식으로 나눌 수 있음

- 1단계 객체 인식 (1-stage detector) : 분류와 위치 검출 두 문제를 동시에 행하는 방법
  → 비교적 빠르지만 정확도가 낮음
- 2단계 객체 인식 (2-stage detector) : 두 문제를 순차적으로 행하는 방법
  → 비교적 느리지만 정확도가 높음

6.2.1 : R-CNN

- R-CNN : 이미지 분류를 수행하는 CNN + 후보 영역 알고리즘
  1. 이미지를 입력으로 받음
  2. 2000개의 바운딩 박스를 선택적 탐색 알고리즘으로 추출한 후 잘라내고, CNN 모델에 넣기 위해 같은 크기 (227 * 227)로 통일한다.
     - 바운딩 박스 : 3D 객체의 형태를 모두 포함할 수 있는 최소 크기의 박스
     - 선택적 탐색 : 객체 인식이나 검출을 위한 가능한 후보 영역을 알아내는 방법
       → 분할 방식을 이용하여 시드(seed)를 선정하고, 그 시드에 대한 완전 탐색을 적용함

     a. [1단계] 초기 영역 생성 (sub-segmentation) : 각각의 객체가 영역 한 개에 할당될 수 있도록 많은 초기 영역을 생성함. (입력된 이미지를 영역 다수 개로 분할하는 과정)

     b. [2단계] 작은 영역의 통합 : 1단계에서 영역 여러 개로 나눈 것들을 비슷한 영역으로 통합
     → 탐욕 알고리즘(결정시 최적이라고 생각되는 것을 선택해 나감)을 사용하여 비슷한 영역이 하나로 통합될 때까지 반복

     c. [3단계] 후보 영역 생성 : 2단계에서 통합된 이미지들을 기반으로 아래와 같이 후보 영역 (바운딩 박스)를 추출함
  3. 크기가 동일한 이미지 2000개에 각각 CNN 모델을 적용함
  4. 각각 분류를 진행하여 결과를 도출함
- R-CNN의 단점
  - 세 단계의 복잡한 학습 과정
  - 긴 학습 시간과 대용량 저장 공간

- 객체 검출 속도 문제

⇒ 이러한 문제를 해결하고자 Fast R-CNN이 생김

6.2.2 : 공간 피라미드 풀링

- 기존 CNN들은 모두 완전연결층을 위해 입력 이미지를 고정해야 했음
  → 신경망을 통과시키려면 이미지를 고정된 크기로 자르거나(crop) 비율을 조정(wrap)했어야 함
- 공간 피라미드 풀링 : 입력 이미지의 크기에 관계없이 합성곱층을 통과시키고, 완전연결층에 전달되기 전에 특성 맵들을 동일한 크기로 조절해 주는 풀링층을 적용하는 기법
  → 이미지의 특징이 훼손되지 않는 특성 맵을 얻을 수 있고, 이미지 분류나 객체 인식 등의 여러 작업에 적용할 수 있다.

6.2.3 : Fast R-NN

- R-CNN의 속도 문제를 개선하고자 RoI 풀링을 도입
  → 선택적 탐색에서 찾은 바운딩 박스 정보가 CNN을 통과하면서 유지되도록 하고 최종 CNN 특성 맵은 풀링을 적용하여 완전연결층을 통과하도록 크기를 조정
  ⇒ 바운딩 박스마다 CNN을 돌리는 시간을 단축할 수 있음
- RoI 풀링 : 크기가 다른 특성 맵의 영역마다 스트라이드를 다르게 최대 풀링 적용 → 결과값 크기를 동일하게 맞춤

6.2.4 : Faster R-CNN

- 기존 Fast R-CNN속도의 걸림돌이었던 후보 영역 생성을 CNN 내부 네트워크에서 진행할 수 있도록 설계함 ⇒ 더욱 빠른 객체 인식을 수행하기 위한 네트워크
- 기존 Fast R-CNN에 후보 영역 추출 네트워크를 추가 (Region Proposal Network, RPN)
  → Faster R-CNN에서는 외부의 느린 선택적 탐색(CPU 계산) 대신 내부의 빠른 RPN(GPU 계산) 사용
  - 바운딩 박스 회귀 (bounding-box regression) : 바운딩 박스가 이미지의 객체를 정확히 포착하여 포함시킬 수 있도록 조정해 주는 역할
- RPN : 후보 영역 추출 네트워크

- 입력 : 특성 맵 N*N 크기의 작은 윈도우 영역
- 해당 영역에 객체의 존재 유무 판단을 위해 binary classification을 수행하는 작은 네트워크 생성
- 위치 보정 (좌표점 추론)을 위해 바운딩 박스 회귀 추가
- 슬라이딩 윈도우 방식으로 앞서 설계한 작은 윈도우 영역을 이용하여 객체 탐색
- 앵커(anchor)
  - 이미지에 존재하는 객체들의 크기와 비율이 다양함
    → 고정된 N*N크기의 입력만으로는 다양한 크기와 비율의 이미지를 수용하기 어려움
  - 이를 보완하기 위해 여러 크기와 비율의 reference box k개를 미리 정의하고 각각의 슬라이딩 윈도우 위치마다 박스 k개를 출력하도록 설계

6.3 : 이미지 분할을 위한 신경망

- 이미지 분할 (image segmentation) : 신경망을 훈련시켜 이미지를 픽셀 단위로 분할하는 것
  → 이미지를 픽셀 단위로 분할하여 이미지에 포함된 객체를 추출함
  - 이미지 분할의 대표적인 네트워크 : 완전 합성곱 네트워크, 합성곱 & 역합성곱 네트워크, U-Net, PSPNet, DeepLabv3/DeepLabv3+

6.3.1 : 완전 합성곱 네트워크

- 완전연결층의 한계
  - 고정된 크기의 입력만 받아들임
  - 완전 연결층을 거친 후에 위치 정보가 사라짐
- 완전 합성곱 네트워크 (Fully Convolutional Network, FCN) : 이미지 분류에서 우수한 성능을 보인 CNN 기반 모델을 변형시켜 이미지 분할에 적합하도록 만든 네트워크
  → 위의 문제를 해결하기 위해 완전연결층을 1 * 1 합성곱으로 대체함
  - 히트맵 : 색상으로 표현할 수 있는 다양한 정보를 이미지 위에 열 분포 형태의 그래픽으로 출력하는 것

6.3.2 : 합성곱 & 역합성곱 네트워크

- 완전 합성곱 네트워크의 단점
  - 여러 단계의 합성곱층과 풀링층을 거치면서 해상도가 낮아짐
  - 낮아진 해상도를 복원하기 위해 업 샘플링 방식 사용 → 이미지의 세부정보들을 잃어버리는 문제 발생
    - 이때의 업 샘플링 : 학습이 불가능, 쌍선형 보간법이나 최근접 이웃 알고리즘을 사용하여 기존 픽셀 사이의 값을 단순 수학 공식으로 채워 넣는 방법
- 합성곱 & 역합성곱 네트워크 (convolutional & deconvolutional network) : 역합성곱 네트워크 도입
  - 역합성곱 : CNN의 최종 출력 결과를 원래의 입력 이미지와 같은 크기로 만들고 싶을 때 사용
    → 시멘틱 분할(segmentic segmentation) 등에 활용 가능
  - 역합성곱을 업 샘플링 이라고도 함
    - 이떄의 업 샘플링 : 필터(가중치, weight)을 가지고 학습이 가능한 방식, 단순 공식으로 채우는 것이 아니라 역전파를 통해 어떻게 확대해야 원본의 세부 정보가 잘 살아나는지를 네트워크 스스로 학습
  - 역합성곱의 작동 방식
    a. 각각의 픽셀 주위에 zero-padding을 추가
    b. 패딩된 것에 합성곱 연산을 수행

6.3.3 : U-Net

- U-Net : 바이오 메디컬 이미지 분할을 위한 합성곱 신경망
  - 속도가 빠르다
    - 기존 슬라이딩 윈도우 방식은 이전 패치(이미지 인식 단위)에서 검증이 끝난 부분을 다음 패치에서 다시 검증함 → 속도가 느림
    - U-Net의 경우 이미 검증이 끝난 패치는 건너뜀 → 속도가 빠름
  - Trade-Off에 빠지지 않는다 : context 인식과 지역화 trade-off 문제 개선

    - 일반적으로 패치 크기가 커지면 넓은 이미지를 인식하는 데 뛰어남
      → context 인식에 탁월함 but 지역화에 한계가 있음
    - context : 이미지의 전체적인 맥락을 파악하는 정보
      → context를 파악하기 위해선 이미지 해상도를 줄이고 수용 영역을 넓혀야 함
    - 지역화(localization) : 물체의 테두리나 정확한 픽셀 위치를 찾아내는 정보
      → localization을 잘하려면 고해상도 픽셀 단위 선명한 세부 정보가 필요
- U-Net의 구조 : FCN 기반 - 수축 경로 + 확장 경로
- U-Net은 수축 경로(Contracting Path, 인코더)와 확장 경로(Expanding Path, 디코더)를 U자 형태로 대칭 배치하고, 그 사이에 스킵 연결(Skip Connection)이라는 다리를 놓았습니다.
  1. **수축 경로 (좌측):** 이미지를 줄여가며 깊은 레이어에서 풍부한 Context 정보(무엇인가?)를 얻어냅니다.
  2. **확장 경로 (우측):** 역합성곱(Transposed Convolution)을 통해 해상도를 다시 키웁니다.
  3. **스킵 연결 :** 수축 경로에서 해상도가 깎이기 전에 갖고 있던 선명한 위치 정보(Localization)를 오른쪽 확장 경로로 그대로 복사해서 전달(Concatenate)합니다.

6.3.4 : PSPNet

- PSPNet : 완전연결층의 한계를 극복하기 위해 피라미드 풀링 모듈을 추가
  - 훈련 과정
    1. 이미지 출력이 서로 다른 크기가 되도록 여러 차례 풀링을 함
       → 각각 다른 크기의 특성 맵은 서로 다른 영역들의 정보를 담음
    2. 1 * 1 합성곱을 사용하여 채널 수를 조정함
       - 풀링층 개수 = N → 출력 채널 수 = 입력 채널 수 / N
    3. 모듈의 입력 크기에 맞게 특성맵을 업 샘플링함 → 양선형 보간법 사용
       - 양선형 보간법 : 화소당 선형 보간을 세 번 수행
         → 새롭게 생성된 화소는 가장 가까운 화소 네 개에 가중치를 곱한 값을 합해서 얻음
    4. 원래의 특성 맵과 1~3과정에서 생성한 새로운 특성 맵들을 병합

6.3.5 : DeepLabv3/DeepLabv3+

- DeepLabv3/DeepLabv3+ : 완전연결층의 단점을 보완하기 위해 Atrous 합성곱 사용
  - 인코더-디코더 구조 + Atrous 합성곱 : 인코더에서 추출된 특성 맵의 해상도를 제어 가능
  - Atrous 합성곱 : 파라미터 수를 늘리지 않으면서도 수용 영역을 크게 키울 수 있음
    - 얼마나 많은 빈 공간을 가질지 결정하는 파라미터 = rate
      → rate r = 1 : 기존 합성곱과 동일, r이 커질수록 빈 공간이 많아짐